

VISÃO COMPUTACIONAL • 2026

# Detecção e Classificação de Golpes em Boxe

*Apresentação de Trabalho*

---

João Pedro Faraoni • Leonardo Marques • Pedro Gimenez • Tom Hunt

# Acesse no Google Slides

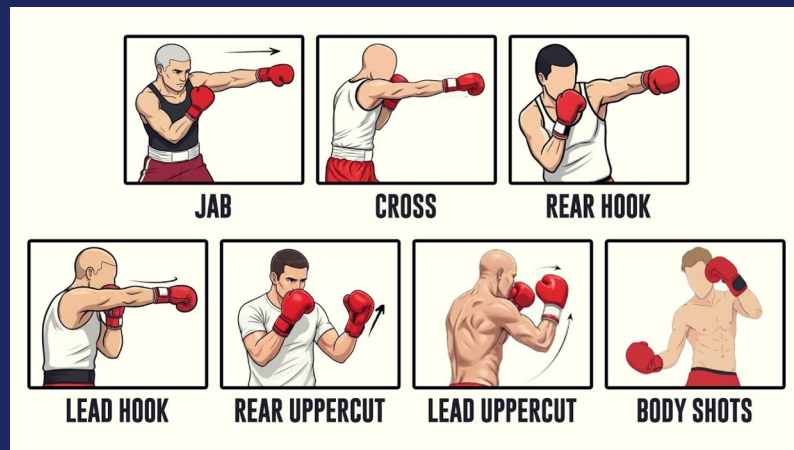
<https://docs.google.com/presentation/d/1SIITKCixfQe4wgktxfAwuUIFhCGcb855GXkrMfx8VBc/edit?usp=sharing>

# Introdução - Boxe e Golpes

- O boxe é se destaca como um esporte de combate caracterizado pela combinação de técnicas ofensivas e defensivas executadas em um ambiente altamente dinâmico [1].

Destacam-se alguns dos golpes:

- **Jab**: golpe rápido com a mão da frente (lead hand), usado para medir distância e preparar ataques.
- **Cross**: golpe reto com a mão de trás (rear hand), geralmente o mais potente.
- **Hook**: golpe lateral curto (gancho), normalmente direcionado à cabeça ou ao corpo.
- **Uppercut**: golpe ascendente, executado de baixo para cima, visando o queixo ou o tronco.



# Revisão da Literatura

## BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization

\*

Rahul Kumar<sup>†a</sup>, Vipul Baghel<sup>†a</sup>, Sudhanshu Singh<sup>a</sup>, Bikash Kumar Badatya<sup>a</sup>, Shivam Yadav<sup>c</sup>,  
Babji Srinivasan<sup>d</sup>, and Ravi Hegde<sup>a</sup>

<sup>a</sup>Department of Electrical Engineering, Indian Institute of Technology Gandhinagar,  
Gandhinagar, India

<sup>c</sup>Department of Computer Science, Dr. A. P. J. Abdul Kalam Technical University, Uttar  
Pradesh, India

<sup>d</sup>Applied Mechanics and Biomedical Engineering, Indian Institute of Technology Madras,  
Tamil Nadu, India

- Paper sobre reconhecimento de ações de boxe.
- Definição do conjunto de classes de golpes
- Janela temporal deslizante de 25 frames para capturar a dinâmica do movimento

## Revisão da Literatura

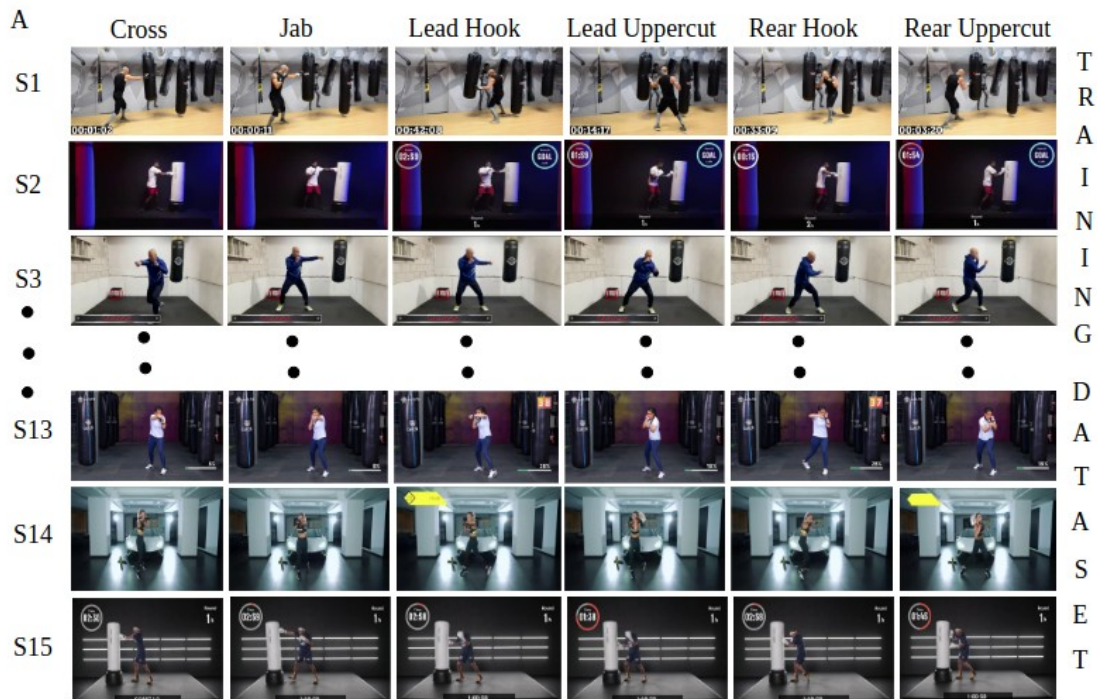
# AlphaPose: Whole-Body Regional Multi-Person Pose Estimation and Tracking in Real-Time

Hao-Shu Fang\*, Jiefeng Li\*, Hongyang Tang, Chao Xu\$, Haoyi Zhu\$, Yuliang Xiu, Yong-Lu Li, and Cewu Lu<sup>†</sup>, *Member, IEEE*

- Paper de pose estimation multi-pessoa em tempo real.
- Normalização de coordenadas dos 17 keypoints COCO.
- Filtragem por confiança média para garantir qualidade dos esqueletos extraídos.
- Base para a extração de keypoints com YOLOv8m-Pose e a estratégia de filtragem por confiança ( $\geq 0.5$ ) usada no pipeline.

# Dataset

Utilizamos o dataset desenvolvido no projeto **BoxingVI** [2] como principal fonte de dados.



# Dataset

O dataset disponibilizado no GitHub [3] conta com marcações sob 10 vídeos individuais de treino de boxe. Para cada vídeo temos o arquivo de anotação em CSV e o arquivo de esqueleto (*skeleton*) do lutador em . npy.

- Para uma janela de no máximo 25 frames, tem-se a marcação em arquivo CSV do frame inicial, frame final e o golpe realizado, para cada vídeo:

| ID | start_frame | end_frame | class     |
|----|-------------|-----------|-----------|
| 1  | 6675        | 6688      | Jab       |
| 2  | 6689        | 6697      | Cross     |
| 3  | 6698        | 6710      | Lead Hook |
| 4  | 6722        | 6734      | Jab       |
| 5  | 6735        | 6745      | Cross     |
| ⋮  | ⋮           | ⋮         | ⋮         |

CSV relativo à V2.

# Dataset

Contabilizando todos os CSVs, temos o total de golpes marcados:

| Classe        | Quantidade |
|---------------|------------|
| Cross         | 1373       |
| Jab           | 1289       |
| Lead Hook     | 1037       |
| Lead Uppercut | 689        |
| Rear Hook     | 394        |
| Rear Uppercut | 681        |

*Distribuição de golpes.*

- Discrepância entre a quantidade de golpes marcados para Jab, Cross e Lead Hook (+1000) quando comparados aos demais.



# Dataset

Os esqueletos são armazenados em arquivos NumPy (.npy) contendo tensores de dimensão (N, T, J, C), onde N representa o número de golpes anotados, T o número máximo de frames por golpe, J o número de articulações do modelo corporal COCO e C as coordenadas espaciais de cada articulação.

| Keypoint  | x     | y     |
|-----------|-------|-------|
| Nose      | 0.346 | 0.295 |
| Left Eye  | 0.343 | 0.3   |
| Right Eye | 0.343 | 0.278 |
| ⋮         | ⋮     | ⋮     |

*Exemplo de coordenadas normalizadas.*

Por exemplo, o arquivo V1.npy possui dimensão (1866, 25, 17, 2), indicando 1866 golpes segmentados, cada um representado por uma sequência temporal de até 25 frames contendo 17 articulações descritas pelas coordenadas normalizadas (x, y).

# Modelagem de Dados

Devido à discrepância entre anotações de golpes como Rear Hook e Rear Uppercut, nossos primeiros resultados não foram satisfatórios. Optamos por agrupar os golpes nas seguintes classes:

| Classe   | Conjunto de Dados                  | Quantidade |
|----------|------------------------------------|------------|
| Straight | Jab $\cup$ Cross                   | 2662       |
| Uppercut | Lead Uppercut $\cup$ Rear Uppercut | 1370       |
| Hook     | Lead Hook $\cup$ Rear Hook         | 1431       |

*União de dados para criação do modelo.*

- Heurística de guarda: se a perna direita está à frente da esquerda, a mão direita é a “mão da frente” e a esquerda a “mão de trás” (e vice-versa).
- Golpes com a “mão da frente”: Jab, Lead Hook, Lead Uppercut
- Golpes com a “mão de trás”: Cross, Rear Hook, Rear Uppercut
- Na pipeline final, basta verificar a posição do lutador e classificar o golpe em lead/rear.

# Modelo de Machine Learning

| Camada               | Parâmetros            | Função                                                                                   |
|----------------------|-----------------------|------------------------------------------------------------------------------------------|
| Entrada              | (25, 102)             | Sequência temporal de poses e derivadas — 102 features (posição, aceleração, velocidade) |
| BiLSTM               | 64 unidades           | Modelagem das dependências temporais em ambas as direções                                |
| Dropout              | 0.3                   | Redução de sobreajuste                                                                   |
| Multi-Head Attention | 2 cabeças, key_dim=32 | Ênfase nos frames mais relevantes do golpe                                               |
| Residual + LayerNorm | —                     | Estabilização do treinamento                                                             |
| Global Avg Pooling   | —                     | Agregação temporal da sequência                                                          |
| Dense                | 64, ReLU              | Extração de características discriminantes                                               |
| Dropout              | 0.3                   | Regularização adicional                                                                  |
| Saída                | Softmax (3 classes)   | Classificação do golpe                                                                   |

Arquitetura da rede proposta para classificação dos golpes.

# Modelo de Machine Learning

Código Python, desenvolvido com a biblioteca TensorFlow 2.21:

```
def build_model(input_shape, nc):  
    inp = Input(shape=input_shape)  
  
    x = Bidirectional(  
        LSTM(64, return_sequences=True, dropout=0.3, recurrent_dropout=0.2)  
    )(inp)  
    x = Dropout(0.3)(x)  
    att = MultiHeadAttention(num_heads=2, key_dim=32)(x, x)  
    x = Add()([x, att])  
    x = LayerNormalization()(x)  
    x = GlobalAveragePooling1D()(x)  
    x = Dense(64, activation="relu", kernel_regularizer=l2(5e-4))(x)  
    x = Dropout(0.3)(x)  
  
    return Model(inp, Dense(nc, activation="softmax")(x))
```

*Arquitetura proposta para o classificador de golpes baseada em Bi-LSTM.*

# Treinamento do Modelo

- **Treino:** anotações e esqueletos dos vídeos V1, V2, V3, V4, V7 e V8, com split 65% / 35% (treino/validação) aleatório.
- **Conjunto de treino:** 2.513 amostras — 1.390 Straight, 673 Hook, 450 Upper-cut.
- **Conjunto de validação:** 1.354 amostras — 749 Straight, 362 Hook, 243 Upper-cut.
- **Conjunto de teste:** vídeos V5, V9 e V10 — 519 Straight, 172 Hook, 208 Upper-cut.
- O vídeo V6 não foi utilizado devido a problemas de formatação nos dados disponibilizados.

# Técnica de Aumento de Dados

Com o objetivo de aumentar a variabilidade do conjunto de treinamento e reduzir o risco de sobreajuste, foi utilizada uma estratégia de espelhamento horizontal dos esqueletos: as coordenadas x são invertidas e as articulações esquerda/direita são trocadas, preservando a consistência anatômica. Após o espelhamento, as mesmas etapas de normalização e extração de características são aplicadas, gerando uma amostra sintética para cada golpe do conjunto de treinamento.

| Classe       | Original    | Espelhado   | Total       |
|--------------|-------------|-------------|-------------|
| Straight     | 1390        | 1390        | 2780        |
| Hook         | 673         | 673         | 1346        |
| Uppercut     | 450         | 450         | 900         |
| <b>Total</b> | <b>2513</b> | <b>2513</b> | <b>5026</b> |

*Impacto da estratégia de aumento de dados por espelhamento horizontal.*

# Resultados: Treinamento do Modelo

O modelo foi treinado utilizando pesos de classe para mitigar o desbalanceamento, com decaimento de taxa de aprendizado e parada antecipada (Early Stopping).

- **EarlyStopping**: paciência de 15 épocas monitorando val\_accuracy.
- **ReduceLROnPlateau**: fator 0.5, paciência de 5 épocas sob val\_loss.
- **ModelCheckpoint**: salvamento exclusivo do melhor estado (best\_model.keras).

O treinamento foi interrompido na época 48 pelo critério de parada (15 épocas sem novo recorde de val\_accuracy), restaurando os pesos da época 33 — a de melhor acurácia de validação (85,16%).

| Conjunto               | Acurácia | Loss   | Val. Acurácia | Val. Loss |
|------------------------|----------|--------|---------------|-----------|
| Final (ép. 48)         | 81.38%   | 0.8162 | 83.53%        | 0.4687    |
| Melhor Estado (ép. 33) | 79.39%   | 0.8769 | 85.16%        | 0.4470    |

*Métricas obtidas no final do treinamento (época de convergência vs. última época).*

# Resultados: Avaliação no Conjunto de Teste

A avaliação final foi realizada de forma estrita no conjunto de teste independente (Vídeos V5, V9 e V10), totalizando 899 golpes anotados.

| Classe         | Precisão | Recall | F1-Score | Suporte |
|----------------|----------|--------|----------|---------|
| Hook           | 0.68     | 0.69   | 0.68     | 172     |
| Straight       | 0.91     | 0.91   | 0.91     | 519     |
| Uppercut       | 0.76     | 0.76   | 0.76     | 208     |
| Acurácia Geral |          |        | 0.83     | 899     |
| Macro Avg      | 0.78     | 0.78   | 0.78     | 899     |
| Weighted Avg   | 0.83     | 0.83   | 0.83     | 899     |

Relatório de classificação obtido no conjunto de teste.

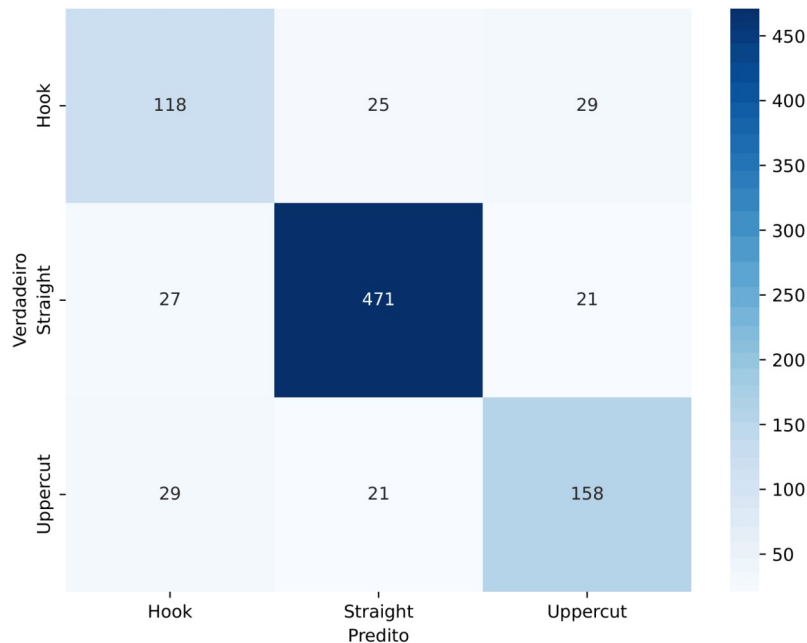
- Destaque: a classe Straight (Jab/Cross) obteve o maior F1-Score (0.91), impulsionada pelo maior volume de dados históricos.
- A classe Uppercut apresentou alto recall (0.76), indicando baixa taxa de falsos negativos.



# Resultados: Matriz de Confusão

Análise das predições versus os rótulos verdadeiros obtidos no conjunto de teste:

- O modelo demonstra solidez na identificação de trajetórias retilíneas (Straight: 471 de 519 corretos).
- O erro mais frequente é Hook predito como Uppercut (29 casos), o que reduz a precisão da classe Hook (0.68).
- O Hook é a classe mais difícil (recall 0.69): parte é confundida com Straight (25) e com Uppercut (29).
- As confusões concentram-se em golpes sem velocidade máxima, cujas assinaturas de aceleração ficam ambíguas.



Matriz de confusão resultante.

# Tracking: Pipeline Multi-Pessoa

1

## Extração multi-pessoa

YOLOv8m-Pose com rastreamento; guardam-se todas as detecções de cada frame (17 keypoints COCO normalizados, confiança por junta e caixa delimitadora em pixels). Mantém-se o filtro de confiança média  $\geq 0.5$ .

2

## Atribuição de identidade

Cada detecção é ligada a um dos dois lutadores por track ID (sinal persistente), com a posição horizontal como fallback quando o ID falta — formando uma sequência de esqueleto por lutador.

3

## Preenchimento de lacunas (gap-fill)

Frames em que um lado não é detectado repetem a última pose conhecida, mantendo a sequência contínua exigida pela janela de 25 frames.

4

## Classificação por lutador

O mesmo classificador por evento (gatilho de Schmitt sobre a velocidade do punho + média do softmax no pico) é executado para cada sequência. Reutilizamos integralmente o modelo treinado, sem nenhum retreino.

# Identidade Híbrida: Track ID + Posição

A identidade é mantida em duas fases por frame: o track ID (ByteTrack) é o sinal primário e PERSISTE mesmo quando os lutadores cruzam de lado; a posição horizontal é só fallback quando o ID falta (occlusão / ID novo). Cada put() registra o ID no slot, então o slot “lembra” o lutador.

```
slot_tid = [None, None]    # ID dono de cada slot (persistente)

# FASE 1: match por track ID (persistencia)
for d in dets:
    for si in (0, 1):
        if slot_tid[si] == d["tid"]:
            put(si, d)      # mesmo lutador -> mesmo slot

# FASE 2: fallback por POSICAO p/ slots livres
livres = sorted(restantes, key=center_x)
put(0, livres[0])         # mais a esquerda
put(1, livres[-1])        # mais a direita
```

*Atribuição em duas fases: ID do rastreador, posição como fallback.*

## Técnicas: Persistência e Modelo Esquerda/Direita

- **ByteTrack** (rastreador por trás do `.track()`): casa as detecções com as trilhas existentes em duas etapas — primeiro as caixas de alta confiança, por sobreposição (IoU) e predição de movimento (filtro de Kalman); depois recupera as de baixa confiança que seriam descartadas. É esse casamento robusto que produz o track ID estável no qual a persistência se apoia.
- **Modelo geométrico esquerda/direita:** quando o ID não basta, dois slots são definidos pela ordem do `center_x` das caixas; a continuidade entre frames usa a menor distância ao último centro conhecido (`last_cx`), e não um limiar fixo no meio da tela — assim o lado atribuído não “salta” sob movimento.
- **Por que combinamos as duas:** o ID sozinho falha em oclusão e movimento rápido (some/troca); a posição sozinho inverte os lados num cruzamento. O híbrido — ID primário, posição como fallback — cobre os dois modos de falha de uma vez.

## Rótulos, Saídas e Robustez

- Rótulos no vídeo: “Boxer 1: Jab (98%)” à esquerda e “Boxer 2: Cross (66%)” à direita, simultaneamente. Com um único lutador em quadro, o rótulo é apenas “Boxer”.
- Limiar de presença mínima (10% dos frames): descarta o “lutador fantasma” gerado por detecções espúrias (reflexo, sombra, treinador) que o YOLO ocasionalmente reporta como 2 persons em poucos frames.
- Dois vídeos de saída: (1) apenas as caixas delimitadoras; (2) caixas + esqueleto COCO desenhado por lutador.

Execução via CLI, gerando os dois vídeos automaticamente:

```
python3 boxe.py --video your-video.mp4 --output outputs/output_name
```

## Inferência do Golpe: Lead vs. Rear

- O modelo prevê só o tipo (Straight, Hook, Uppercut); a mão do golpe (lead/rear) é decidida na pipeline, por geometria pura sobre os keypoints — sem um segundo modelo treinado.
- Stance estática (comparar a posição dos tornozelos) é ruído: o YOLO “alucina” o lado ocluído do corpo e a câmera lateral colapsa a profundidade frente/trás na projeção 2D.
- Decisão por golpe: o punho com maior deslocamento líquido é a mão que golpeou (desempate pelo alcance punho-ombro); a extensão define a “frente” local, e a perna mais avançada nessa direção é o pé da frente — mão do mesmo lado do pé da frente → lead; caso contrário → rear.

Mapeamento direto para as 6 classes finais:

- **Straight** → Jab (lead) ou Cross (rear)
- **Hook / Uppercut** → prefixo Lead ou Rear

## Inferência Lead/Rear: Resultados

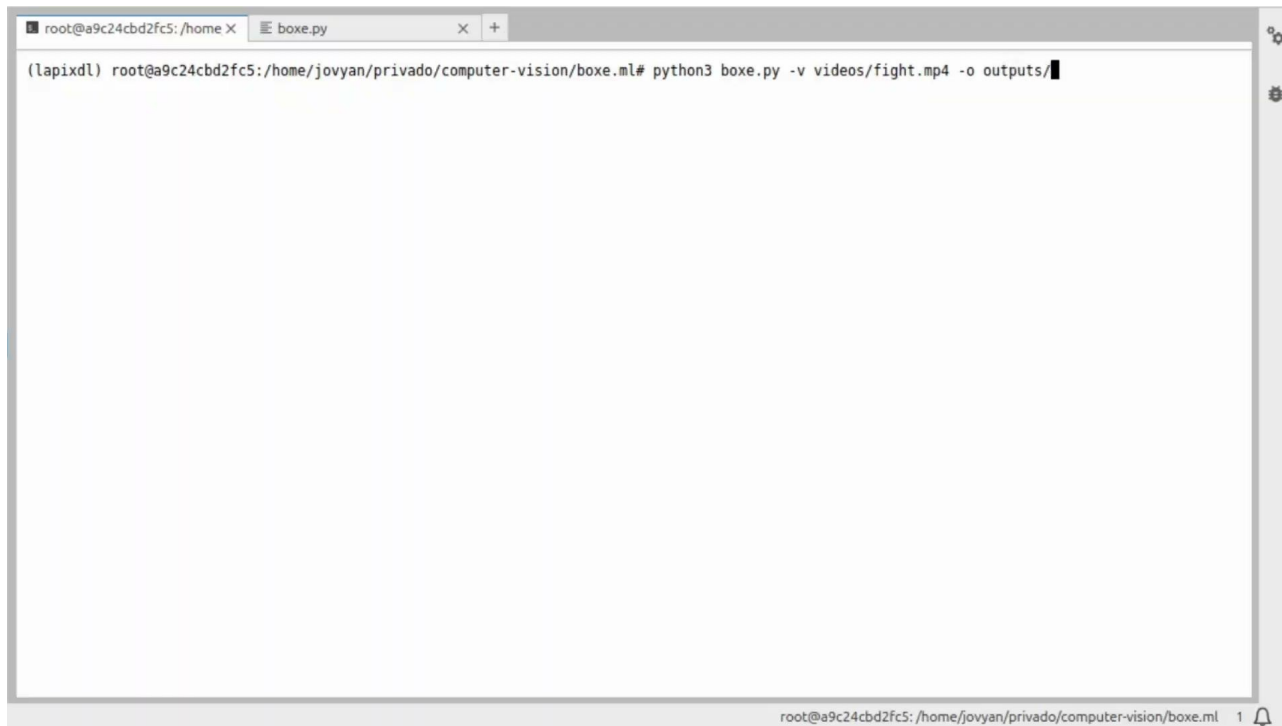
- A decisão usa o esqueleto sem suavização temporal: um soco é ida-e-volta em poucos frames e a média móvel achata o deslocamento do punho que golpeou (a acurácia cai de 0,89 para 0,28 com janelas suavizadas).
- BoxingVI: 0,85 no treino · 0,77 na validação cruzada entre vídeos · 0,74 no teste.
- Vídeo gravado pelo grupo (18 golpes anotados manualmente, câmera traseira em ângulo de  $\frac{3}{4}$ ):  $16/18 = 0,89$  por segmento anotado — a geometria generaliza para pontos de vista fora do dataset.

Avaliação end-to-end reproduzível contra o ground truth anotado:

```
python3 evaluation/eval_leadrear.py
```

# Demonstração: Execução via CLI

OBS: Vídeo acelerado e clipado, para fins de tempo de apresentação.

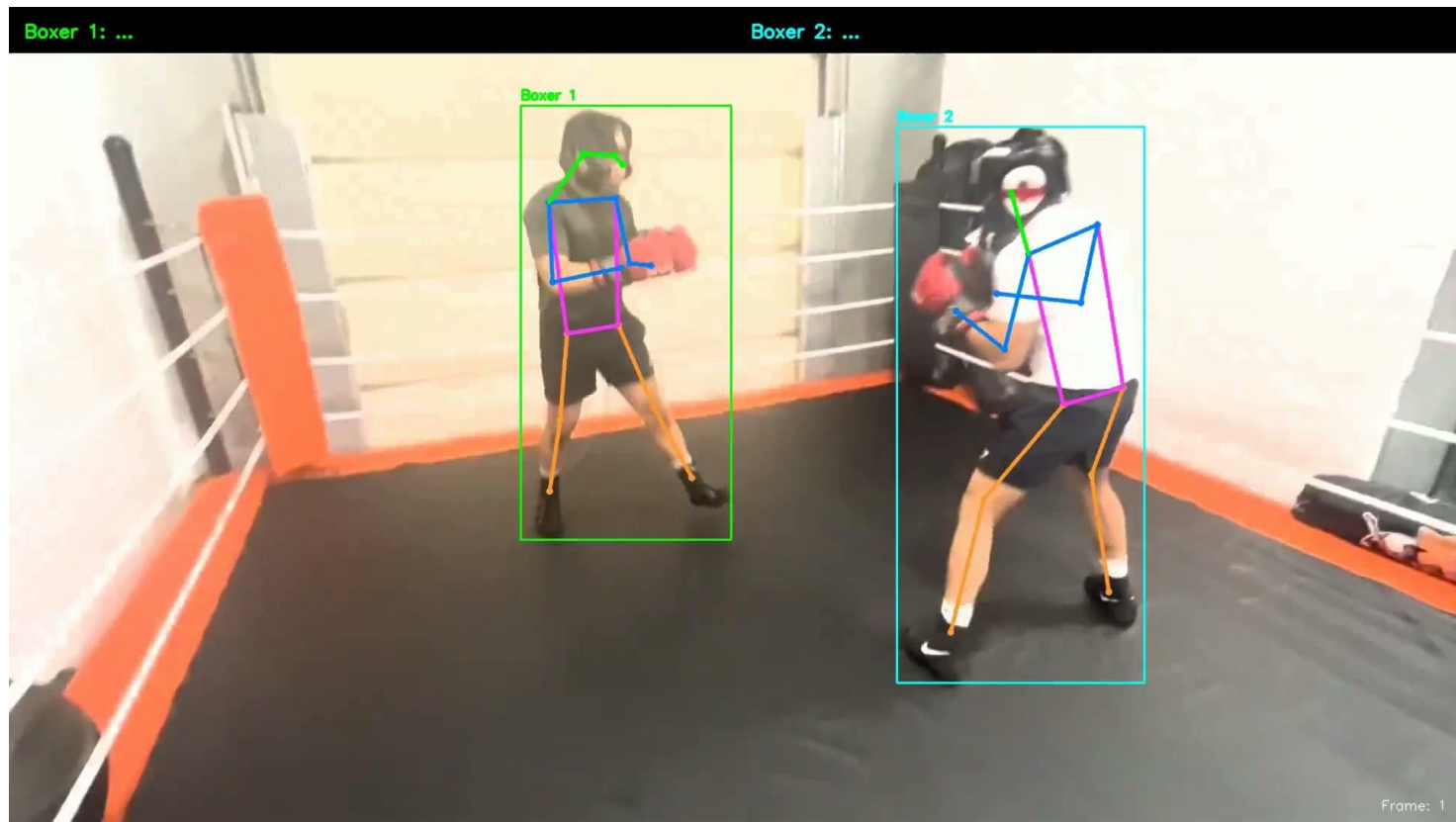


A terminal window with a title bar containing two tabs: 'root@a9c24cbd2fc5: /home' and 'boxe.py'. The terminal content shows a command being executed: '(lapixdl) root@a9c24cbd2fc5:/home/jovyan/privado/computer-vision/boxe.ml# python3 boxe.py -v videos/fight.mp4 -o outputs/'. The command is followed by a cursor. The terminal has a light gray background and a dark gray border. On the right side of the terminal window, there are two gear icons for settings. At the bottom of the terminal window, the status bar shows the current directory and file: 'root@a9c24cbd2fc5: /home/jovyan/privado/computer-vision/boxe.ml' followed by a page number '1' and a bell icon.

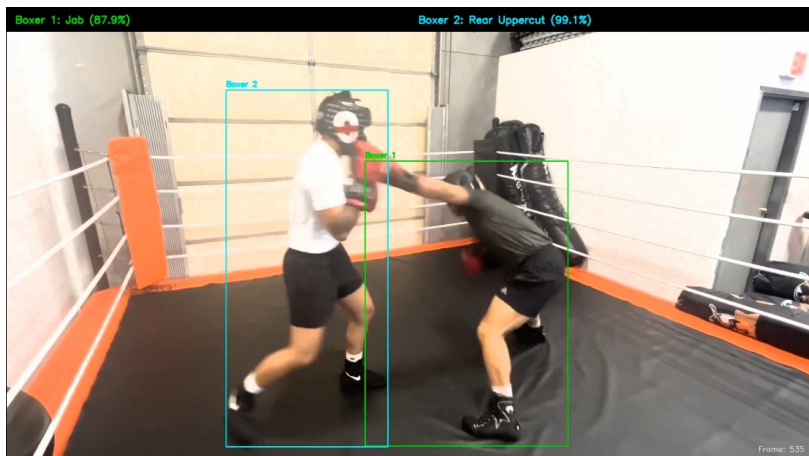
```
(lapixdl) root@a9c24cbd2fc5:/home/jovyan/privado/computer-vision/boxe.ml# python3 boxe.py -v videos/fight.mp4 -o outputs/
```



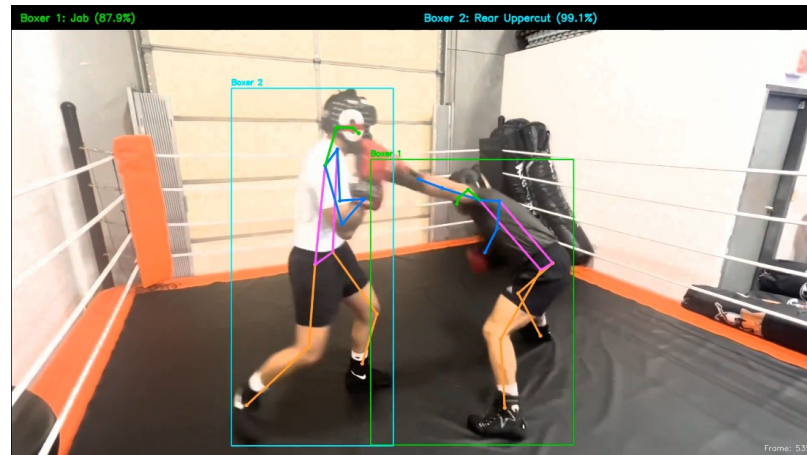
# Demonstração: Resultado Final



# Demonstração: Spar com Dois Lutadores



Dois lutadores rastreados — rótulo de golpe por id.



Cena com o esqueleto COCO sobreposto.

# Referências

- [1] F. Loio Pinto et al., «The efficiency and transversality of traditional boxing skills to several full-contact combat sports: a narrative review», Archives of Budo Science of Martial Arts and Extreme Sports, vol. 18, pp. 1-10, 2023.
- [2] R. Kumar et al., «BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization». [Em linha]. Disponível em: <https://arxiv.org/abs/2511.16524>
- [3] R. Kumar et al., «BoxingVI: A Multi-Modal Benchmark for Boxing Action Recognition and Localization». [Em linha]. Disponível em: <https://github.com/Bikudebug/BoxingVI>

# Vídeo Apresentação



O vídeo da apresentação destes slides pode ser encontrado neste QR Code ou por este link: <https://hlnks.co/64876acd>

## Repositório no GitHub



O repositório do projeto está disponível no GitHub e pode ser encontrado neste QR Code ou por este link: <https://github.com/leonardosm14/boxe.ml>